**Nutan Maharashtra Vidya Prasarak Mandal's**

# NUTAN MAHARASHTRA INSTITUTE OF ENGINEERING & TECHNOLOGY

Samarth Vidya Sankul, Vishnupuri, Talegaon Dabhade, Pune - 410507



**DEPARTMENT OF COMPUTER APPLICATION (BCA & MCA)**

---

# MINI PROJECT REPORT

**ON**

# "TESTORA – SMART APTITUDE PREPARATION AND ONLINE TEST SYSTEM"

Submitted in partial fulfillment of the requirements for the course

**MINI PROJECT**

| | |
|---|---|
| **Group No.** | [Group No.] |
| **Student Name(s)** | [Student 1, Student 2, Student 3] |
| **Roll No. / PRN / Exam Seat No.** | [Enter details] |
| **Class / Division** | MCA – [A/B] |
| **Project Guide** | [Guide Name] |

**ACADEMIC YEAR 2026-27**

**Nutan Maharashtra Vidya Prasarak Mandal's**

# NUTAN MAHARASHTRA INSTITUTE OF ENGINEERING & TECHNOLOGY

Samarth Vidya Sankul, Vishnupuri, Talegaon Dabhade, Pune - 410507

# CERTIFICATE

Department of Computer Application

This is to certify that **[Student 1, Student 2, Student 3]**, Roll No./PRN/Exam Seat No. **[Enter details]**, of MCA has satisfactorily completed the Mini Project entitled **"Testora – Smart Aptitude Preparation and Online Test System"** under the guidance of **[Guide Name]** during the Academic Year 2026-27. The work presented in this report is approved for submission as part of the Mini Project requirements.

**Project Guide** **Project Coordinator** **Head of Department** **Principal**

Date: ____ / ____ / 2026

# STUDENT DECLARATION

We hereby declare that the Mini Project report entitled “Testora – Smart Aptitude Preparation and Online Test System” submitted to the Department of Computer Application, Nutan Maharashtra Institute of Engineering & Technology, is an original record of the work carried out by us under the guidance of [Guide Name] during the Academic Year 2026-27.

We further declare that this work has not been submitted, either in full or in part, to any other university or institute for the award of any degree, diploma, or certificate. All sources of information used in this project have been properly acknowledged and cited.

We understand that any form of plagiarism, fabrication of results, or misrepresentation of work may lead to disciplinary action as per institute rules.

| Sr. No. | Student Name | Roll No. / PRN | Signature |
|---|---|---|---|
| 1 | | | |
| 2 | | | |
| 3 | | | |

Place: _____ Date: ____ / ____ / 2026

# ACKNOWLEDGEMENT

We take this opportunity to express our sincere gratitude to everyone who supported us during the development of our Mini Project, “Testora – Smart Aptitude Preparation and Online Test System”.

We are thankful to Nutan Maharashtra Institute of Engineering & Technology, Talegaon Dabhade, Pune, for providing the academic environment and the laboratory facilities needed to complete this work. We express our respectful thanks to the Principal for encouraging students to undertake practical projects, and to the Head of the Department of Computer Application for continuous support and guidance to the students of the department.

We are deeply grateful to our Project Guide, [Guide Name], for the valuable guidance, regular reviews and constructive suggestions given at every stage of the project, from problem identification and requirement analysis to design, implementation and testing. The corrections suggested during the progress presentations helped us to improve both the system and this report. We also thank the Project Coordinator for organising the schedule and the review sessions of the Mini Project.

We thank all the faculty members and the non-teaching staff of the department for their cooperation and for the facilities provided to us. Our sincere thanks are due to our classmates, who tested the application, gave honest feedback on its usability and shared useful ideas. We also acknowledge the open-source communities behind React, Node.js, Express, MongoDB and the other tools used in this project, whose documentation made it possible for us to learn and build quickly.

Finally, we thank our families for their patience, encouragement and moral support throughout the course of this work.

**[Student 1, Student 2, Student 3]**

# ABSTRACT

Preparing for campus placement examinations requires regular practice of aptitude questions, but students usually depend on scattered websites that offer static question lists, while teachers and colleges have no simple way to set their own timed tests, assign them to students and measure the results. This project, Testora, is a web-based aptitude preparation and online test system that brings learning, practice, assessment and analytics together on one platform for three types of users: students, teachers and administrators.

The objective was to build a secure, role-based application in which content is organised as Category, Topic and Question; students can study concept notes, practise topic-wise questions with instant feedback and attempt timed tests; teachers can create and assign quizzes; and administrators can manage users and content. Testora is implemented as a single-page application using React, Vite and Tailwind CSS, a REST API built with Node.js and Express, and a MongoDB database accessed through Mongoose. Authentication uses JSON Web Tokens and bcrypt-hashed passwords, and every protected route is checked for both login status and role.

The major features include timed mock tests created by administrators; teacher quizzes created through a three-step workflow that generates question forms dynamically and assigns the quiz to all active students; server-side evaluation with configurable marks per question and optional negative marking; correct answers that are never sent to the browser before submission; immediate blocking of deactivated accounts; topic-wise performance analytics that flag weak topics; validated JSON import of questions with duplicate detection and no partial imports; and CSV export.

The system was verified with forty-one automated API test cases and a scripted walkthrough of every user role in a real browser; all forty-one test cases passed on an isolated test database. The results show that the objectives were met and that the platform can serve as a practical, low-cost aptitude practice and assessment tool for a college. Future work includes restoring interrupted mock tests, PDF export, question randomisation, proctoring support and an automated regression suite.

**Keywords:** Aptitude Assessment, Online Test System, MERN Stack, Role-Based Access Control, Performance Analytics

# TABLE OF CONTENTS


| Section | Title | Page No. |
|---|---|---|
| | Certificate | 2 |
| | Student Declaration | 3 |
| | Acknowledgement | 4 |
| | Abstract | 5 |
| | List of Figures | 7 |
| | List of Tables | 8 |
| | List of Abbreviations | 9 |
| 1 | Introduction | 10 |
| 2 | Literature Survey & Existing System | 14 |
| 3 | Requirement Analysis & System Design | 17 |
| 4 | Implementation | 29 |
| 5 | Testing & Results | 40 |
| 6 | Conclusion & Future Scope | 45 |
| | References | 47 |
| | Appendices | 49 |

# LIST OF FIGURES


| Figure No. | Figure Caption | Page No. |
|---|---|---|
| Fig. 3.1 | System Architecture of Testora | 20 |
| Fig. 3.2 | Use Case Diagram – Student Module | 21 |
| Fig. 3.3 | Use Case Diagram – Teacher Module | 21 |
| Fig. 3.4 | Use Case Diagram – Admin Module | 22 |
| Fig. 3.5 | Data Flow Diagram – Level 0 (Context Diagram) | 22 |
| Fig. 3.6 | Data Flow Diagram – Level 1 | 23 |
| Fig. 3.7 | Entity-Relationship Diagram of Testora | 24 |
| Fig. 4.1 | Test Attempt and Evaluation Flowchart | 33 |
| Fig. 4.2 | Login Screen | 34 |
| Fig. 4.3 | Student Dashboard | 34 |
| Fig. 4.4 | Topic Detail with Concept Cards | 35 |
| Fig. 4.5 | Timed Exam Screen with Question Palette | 35 |
| Fig. 4.6 | Result Screen | 35 |
| Fig. 4.7 | Performance Analysis | 36 |
| Fig. 4.8 | Create Quiz – Step 2: Quiz Details | 36 |
| Fig. 4.9 | Create Quiz – Step 3: Set Questions | 36 |
| Fig. 4.10 | Admin Dashboard | 37 |
| Fig. 4.11 | Admin – Import Questions via JSON | 37 |
| Fig. 4.12 | Request Authentication and Authorization Pipeline | 38 |

# LIST OF TABLES


| Table No. | Table Title | Page No. |
|---|---|---|
| Table 2.1 | Comparison of Existing Systems with Testora | 16 |
| Table 3.1 | Functional Requirements of Testora | 17 |
| Table 3.2 | Non-Functional Requirements of Testora | 18 |
| Table 3.3 | Hardware and Software Requirements | 18 |
| Table 3.4 | User Collection | 25 |
| Table 3.5 | Category and Topic Collections | 25 |
| Table 3.6 | Concept Collection | 25 |
| Table 3.7 | Question Collection | 25 |
| Table 3.8 | MockTest Collection | 26 |
| Table 3.9 | TeacherQuiz Collection | 26 |
| Table 3.10 | QuizAssignment Collection | 26 |
| Table 3.11 | TestAttempt Collection | 26 |
| Table 3.12 | Module Overview | 27 |
| Table 4.1 | Development Environment | 29 |
| Table 4.2 | Topic Classification Rules | 33 |
| Table 4.3 | Environment Variables of the Server | 37 |
| Table 4.4 | Module Inputs and Outputs | 39 |
| Table 5.1 | Test Cases and Results | 40 |
| Table 5.2 | Test Results by Module | 42 |
| Table 5.3 | Objectives and Outcome | 43 |
| Table 5.4 | Handling of Invalid Inputs and Exceptions | 43 |
| Table A.1 | Demonstration Accounts | 49 |
| Table B.1 | Test Evidence by Module | 50 |

# LIST OF ABBREVIATIONS

| Abbreviation | Full Form |
|---|---|
| API | Application Programming Interface |
| CORS | Cross-Origin Resource Sharing |
| CRUD | Create, Read, Update, Delete |
| CSS | Cascading Style Sheets |
| CSV | Comma-Separated Values |
| DFD | Data Flow Diagram |
| ER | Entity-Relationship |
| FR | Functional Requirement |
| HTTP | Hypertext Transfer Protocol |
| JSON | JavaScript Object Notation |
| JWT | JSON Web Token |
| MCQ | Multiple-Choice Question |
| MERN | MongoDB, Express.js, React, Node.js |
| ODM | Object Document Mapper |
| OWASP | Open Worldwide Application Security Project |
| PDF | Portable Document Format |
| RBAC | Role-Based Access Control |
| REST | Representational State Transfer |
| SPA | Single-Page Application |
| TC | Test Case |
| UI | User Interface |
| XSS | Cross-Site Scripting |

# CHAPTER 1
# INTRODUCTION

## 1.1 Background / Overview

Aptitude tests covering quantitative ability, logical reasoning and verbal ability are a standard first round in campus placements and in many competitive examinations. Candidates are expected to solve a large number of questions accurately inside a strict time limit. Success depends less on one-time learning and more on regular topic-wise practice, repeated timed test-taking and honest feedback about weak areas.

Colleges support this preparation through training sessions and teacher-made tests, while students rely mostly on public websites, books and commercial test-series platforms. These resources are useful, but they are separate from one another and from the college. A student reads concepts in one place, practises on another website and takes mock tests on a third, and no single system records progress or lets the college's own teachers set, assign and monitor tests. A web-based platform that integrates these activities, and that the institution itself controls, addresses a clear need.

## 1.2 Problem Statement

Students preparing for aptitude examinations do not have one integrated, institution-controlled platform that combines concept learning, topic-wise practice, timed tests with fair scoring and objective feedback on weak topics. Teachers have no simple way to create a timed quiz, assign it to their students and see class results without using heavy learning-management software or manual paper evaluation, and administrators have no unified place to manage users, subjects and the question bank. This project addresses these gaps by building *Testora*, a secure role-based web application for students, teachers and administrators.

## 1.3 Project Objectives

The project was planned around the following measurable objectives:

1. Provide separate, secure portals for students, teachers and administrators using JSON Web Token authentication and server-side role checks on every protected API route.
2. Organise content as Category → Topic → Question, with administrator-managed categories, topics, concept notes and a question bank. The demonstration data set delivered with the project contains 3 categories, 22 topics, 58 questions and 5 mock tests.

3. Deliver timed mock tests and teacher quizzes that are evaluated on the server with configurable marks per question and optional negative marking, and never send correct answers to the browser before submission.
4. Let a teacher create a quiz through a guided three-step workflow (category, details, questions) in which question forms are generated from the chosen question count, and assign the quiz automatically to all active students.
5. Report each student's performance topic-wise and classify topics as Weak, Needs Practice or Strong using a minimum of five attempted questions per topic.
6. Support bulk content management through validated JSON import (at most 2 MB and 500 questions per import, all-or-nothing, duplicate-safe) and CSV export, and verify the system with at least 40 documented test cases.

## 1.4 Scope of the Project

**Included in the scope:**

- Three user roles (student, teacher, administrator) with registration, login, profile management and password change.
- Student features: browsing categories and topics, reading concept notes, untimed practice with instant feedback, timed mock tests and assigned quizzes, results with answer review, performance analytics and test history.
- Teacher features: dashboard, guided quiz creation with dynamic question forms, marks and optional negative marking, automatic assignment, quiz management and class results.
- Administrator features: dashboard counts, student and teacher management (activation and deactivation), categories, topics, concepts, question bank, JSON/CSV import, CSV export and mock test management.
- A responsive single-page web interface backed by a REST API and a MongoDB database.

**Excluded from the scope:**

- PDF export or printing of questions (only CSV export is provided).
- Online proctoring such as webcam monitoring or tab-switch detection.
- Question types other than single-answer multiple-choice questions with four options.
- Payments or subscriptions, native mobile applications, and e-mail or SMS notifications.

- Hard deletion of users: accounts are deactivated so that historical quizzes and results are preserved.
- Manual selection of individual students for a quiz: every quiz is assigned to all active students by design.
- Public hosting: the system was developed and tested in a local development environment against a cloud MongoDB database.

## 1.5 Proposed Solution

Testora is a three-tier web application. A React single-page application provides role-specific portals; a Node.js and Express REST API implements authentication, authorization, validation, test evaluation and analytics; and a MongoDB database stores users, subjects, questions, tests, assignments and attempts. The central idea is that all learning and assessment activity is tied to the Category → Topic → Question hierarchy, so that every answer a student gives can be traced back to a topic. This makes topic-wise feedback possible without any extra effort from teachers.

Tests are evaluated only on the server. The browser receives questions without their correct answers and the correct answers are revealed only in the result review after the attempt has been submitted. Marks are calculated as correct answers multiplied by the marks per question, minus wrong answers multiplied by the negative mark, so that negative marking can be switched on for any test.

## 1.6 Key Features

- **Student portal (12 screens):** dashboard, topics, topic detail with concept cards, practice mode, mock tests, timed exam with question palette and mark-for-review, assigned quizzes, result page with answer review, performance analysis, test history and profile.
- **Teacher portal (5 screens):** dashboard, quiz management, three-step quiz creator with dynamically generated question forms, quiz results and profile with specialization.
- **Admin portal (8 screens):** dashboard with live counts, students, teachers, categories, topics, concepts, questions with import and export, and mock tests with publish control.
- **Marks and negative marking** configurable per mock test and per quiz, with the maximum marks stored with each attempt.
- **Security controls:** bcrypt password hashing, JWT sessions, role-based authorization, immediate rejection of deactivated accounts, input validation, secure HTTP headers and restricted CORS.

- **Data tools:** JSON import with full validation and duplicate detection, CSV import, and CSV export of all or filtered questions.

## 1.7 Organization of the Report

Chapter 1 introduces the problem, objectives, scope and the proposed solution. Chapter 2 reviews the technologies used and existing systems and identifies the development gap. Chapter 3 presents the functional and non-functional requirements, the system architecture, the use case, data flow and entity-relationship diagrams, the database design and the module design. Chapter 4 explains the implementation, the core algorithms, the user interface with screenshots, integration and deployment steps, and the security and validation controls. Chapter 5 describes the testing strategy, the test cases and their results, and the limitations of the current system. Chapter 6 concludes the report and lists the future scope. The References and Appendices (installation guide, test evidence and supporting documents) follow.

# CHAPTER 2
# LITERATURE SURVEY & EXISTING SYSTEM

## 2.1 Literature / Technology Survey

Computer-based assessment replaces paper tests with software that presents questions, measures time, evaluates answers automatically and stores the outcome. For aptitude preparation the most useful properties are immediate evaluation, repeatable timed practice and analysis of results by topic. The following technologies were surveyed and selected to build such a system; together they form the widely used MERN stack (MongoDB, Express.js, React and Node.js).

**React.** React is a component-based JavaScript library for building user interfaces, in which the screen is described as a tree of reusable components that re-render when their state changes [1]. It suits Testora because the three portals share many components (tables, modals, spinners, confirmation dialogs) and because test-taking screens need frequent local state updates such as the running timer and the selected answers.

**Node.js and Express.** Express is a minimal web framework for Node.js whose request handling is built around a pipeline of middleware functions [2]. Testora uses this pipeline for security headers, authentication, role authorization and input validation before a request reaches the business logic. Using JavaScript on both client and server also lets the team work in one language.

**MongoDB and Mongoose.** MongoDB is a document database that stores records as flexible JSON-like documents [3]. Mongoose adds schemas, validation, default values, indexes and object references on top of MongoDB [4]. Questions with embedded option arrays, and attempts with embedded answer and topic-performance arrays, map naturally to documents and avoid many table joins.

**JSON Web Tokens.** A JSON Web Token is a compact, digitally signed token that carries claims between two parties [5]. After login Testora issues a token containing the user identifier; the client sends it with every request and the server verifies the signature. The server additionally re-reads the user record on each request so that a deactivated account is rejected immediately.

**bcrypt.** bcrypt is an adaptive password-hashing scheme that adds a random salt and a configurable work factor, making stored passwords resistant to brute-force and rainbow-table attacks [6]. Testora stores only the bcrypt hash of each password.

**Tailwind CSS, Vite and Recharts.** Tailwind CSS is a utility-first styling framework that makes it easy to build consistent, responsive layouts [7]. Vite is a build tool and

development server that provides fast start-up, hot module replacement and optimised production builds [8]. Recharts provides composable chart components for React and is used for the topic-accuracy chart on the performance screen [9].

**express-validator and Multer.** express-validator supplies validation and sanitisation chains for Express routes, which Testora uses to reject malformed input before it reaches the controllers [10]. Multer is the Express middleware for multipart file uploads and is used, with size and file-type limits, for the CSV and JSON import files [11].

## 2.2 Existing System

Aptitude preparation and testing is currently handled in four main ways. The following observations about public systems are based on the pages listed in the References.

- **Manual and paper-based tests.** Teachers prepare question papers, students answer on paper and the evaluation and result compilation are done by hand. Feedback is slow and topic-wise analysis is rarely produced.
- **Learning-management systems.** The Moodle Quiz activity lets teachers build quizzes inside a course from questions stored in a reusable question bank [12]. It is a module of a Moodle course, so it can be used only after a course has been set up in Moodle.
- **Form-based quizzes.** Google Forms can auto-grade quizzes using an answer key, point values and answer feedback, and its summaries include frequently missed questions and score statistics [13]. Individual grading and score e-mails depend on collecting respondents' e-mail addresses.
- **Free aptitude websites.** Sites such as IndiaBIX publish topic-wise aptitude questions with solved answers and explanations, and offer PDF or e-book downloads [14]. The content is created and controlled by the site owner.
- **Commercial test-series platforms.** Platforms such as Testbook offer large mock-test series with score analysis and a strengths-and-weaknesses breakdown, but many tests are paid and the content is controlled by the vendor [15].

## 2.3 Limitations of Existing System

- No single system joins concept notes, topic-wise practice, timed tests and per-student analytics; students must move between several tools.
- Public aptitude websites and commercial platforms are controlled by their owners, so a college cannot add its own questions, schedule its own tests or monitor its own students.

- General-purpose learning-management systems are organised around courses and typically need hosting and administration, which is more than a department needs for aptitude practice alone.
- Form-based quizzes are simple to create, but the documentation reviewed does not describe a timed exam mode or topic-level analytics [13].
- Paper-based tests need manual evaluation, which delays feedback and makes weak-topic identification difficult.
- Many commercial services require payment, which limits access for students who need regular practice.

## 2.4 Comparative Analysis

Table 2.1 compares the four approaches reviewed with the proposed system. Only features that are stated on the cited pages are listed for the existing systems.

**Table 2.1 Comparison of Existing Systems with Testora**

| System / Reference | Main Features | Technology | Limitations | How Proposed System Improves |
|---|---|---|---|---|
| Moodle Quiz activity [12] | Teacher-built quizzes; reusable question bank; teachers review attempts and grades | Activity module of a Moodle course | Used inside a Moodle course, which must be set up first; negative marking and topic analytics not described on the page reviewed | Purpose-built, lightweight; marks, negative marking and topic analytics available out of the box |
| Google Forms quizzes [13] | Answer-key auto-grading; point values; answer feedback; missed-question and score summaries | Web-based form and quiz feature | Individual grading needs e-mail collection; timed mode and topic analytics not described | Timed exam with question palette, practice mode and topic-wise analytics |
| IndiaBIX aptitude section [14] | Topic-wise questions with solved explanations; PDF and e-book downloads | Public website | Content controlled by the site owner; no per-user analytics shown on the page reviewed | College-owned content; teachers author quizzes; per-student results |
| Testbook [15] | Mock-test series; detailed score analysis; strengths and weaknesses; live tests | Commercial exam-preparation platform | Many tests are paid; content controlled by the vendor; focus on government and entrance exams | Free for the institution; aptitude-focused; teacher-created assigned quizzes |

## 2.5 Identified Research / Development Gap

The review shows that good tools exist for individual activities — question banks, auto-graded quizzes, practice websites and large test series — but none of them gives a college a single, self-controlled system in which administrators manage subjects and content, teachers author and assign timed quizzes, and students learn, practise, take tests and see topic-wise feedback. Testora fills this gap with a small, open and extensible platform built on a common technology stack that the department can host, modify and extend.

# CHAPTER 3
# REQUIREMENT ANALYSIS & SYSTEM DESIGN

## 3.1 Functional Requirements

Requirements are commonly classified as functional requirements, which describe what the system must do, and non-functional requirements, which describe the qualities it must have [16]. The functional requirements of Testora are numbered in Table 3.1.

**Table 3.1 Functional Requirements of Testora**

| ID | Module | Requirement |
|---|---|---|
| FR1 | Authentication | A new student can register with name, e-mail, mobile number, password, college, course and year; the role is always set to student by the server. |
| FR2 | Authentication | Users log in with e-mail and password and receive a signed token; deactivated users cannot log in. |
| FR3 | Authentication | Users can update their profile and change their password after entering the current password. |
| FR4 | Admin | The dashboard shows live counts of students, teachers, topics and questions (plus mock tests and attempts) and recent test attempts. |
| FR5 | Admin | The admin can list students (name, college, course, year, status) and activate or deactivate any student; deactivation blocks all further API access. |
| FR6 | Admin | The admin can create teacher accounts with specialization, edit teachers and activate or deactivate them. |
| FR7 | Admin | The admin can create, edit and deactivate categories and topics; each topic belongs to one category. |
| FR8 | Admin | The admin can create, edit and delete concept notes (explanation, formulas, examples, important points) for a topic. |
| FR9 | Admin | The admin can create, edit and deactivate questions with four options, one correct answer and an optional explanation, and can filter by category and topic and search by text. |
| FR10 | Admin | The admin can import questions from a JSON file or pasted JSON; the whole import is rejected if any item is invalid or a duplicate. |
| FR11 | Admin | The admin can import questions from CSV and export all or filtered questions to CSV. |
| FR12 | Admin | The admin can create, edit, delete, publish and unpublish mock tests with duration, marks per question and optional negative marking. |
| FR13 | Teacher | The teacher creates a quiz in three steps: select category, enter details (title, description, question count, duration, marks, optional negative marking, dates) and set questions. |
| FR14 | Teacher | The system generates one question form per requested question; each question needs a topic, text, four options, a correct answer and an optional explanation. |
| FR15 | Teacher | A created quiz is assigned automatically to all active students; no individual student selection is required. |
| FR16 | Teacher | The teacher can list and delete own quizzes and view each quiz’s results, completion count and average accuracy. |
| FR17 | Student | The student can browse categories and topics and read concept notes for a topic. |
| FR18 | Student | The student can practise topic questions one at a time and receive immediate correctness feedback and explanation. |
| FR19 | Student | The student can attempt a published mock test once within its time limit, using a question palette, mark-for-review and automatic submission on timeout. |
| FR20 | Student | The student can attempt an assigned quiz only inside its start–end date window and only once. |

| ID | Module | Requirement |
|---|---|---|
| FR21 | Student | The server evaluates every attempt and stores marks obtained, maximum marks, counts of correct, wrong and skipped answers, accuracy and topic-wise performance. |
| FR22 | Student | The result page shows marks, accuracy, time taken, topic performance and an answer review that reveals correct answers only after submission. |
| FR23 | Student | The student can view performance analytics with weak-topic recommendations and a filterable, paginated test history. |
| FR24 | System | All write requests are validated on the server and invalid input is rejected with a clear message. |
| FR25 | System | Every protected route checks the user's role; correct answers are never included in question payloads sent before submission. |

## 3.2 Non-Functional Requirements

Table 3.2 lists the non-functional requirements. The security requirements are aligned with the guidance of the OWASP Top 10 [17].

**Table 3.2 Non-Functional Requirements of Testora**

| Category | Requirement |
|---|---|
| Performance | List endpoints for students and questions are paginated; queries populate only the references they need; the production client bundle is about 778 kB minified (about 217 kB gzipped). |
| Security | Passwords are stored as bcrypt hashes; sessions use signed JWTs with a 7-day default expiry; role authorization on every protected route; deactivated accounts rejected on each request; secure HTTP headers (helmet); CORS restricted to the client origin; input validation; correct answers withheld before submission. These controls follow the principles in the OWASP Top 10 [17]. |
| Usability | Role-specific navigation; a guided three-step quiz creator; confirmation dialogs for destructive actions; loading, empty, success and error states; toast notifications. |
| Reliability | Test evaluation is performed on the server and attempts are persisted; quiz creation and JSON import validate the whole request before any write; uploaded temporary files are deleted after parsing; a central error handler returns JSON error messages. |
| Scalability | The API is stateless (JWT), so several server instances can run behind a load balancer; MongoDB Atlas can be scaled independently; code is split into routes, controllers, services and models. |
| Compatibility | Works in current Chrome, Edge and Firefox on desktop and mobile widths (layouts verified at 1280 px and 390 px); requires Node.js 18 or later. |
| Maintainability | Separation of concerns, reusable React components, documented set-up in the README and in this report. |

## 3.3 Hardware and Software Requirements

Table 3.3 gives the minimum hardware and the software, frameworks and libraries needed to build and run the system.

**Table 3.3 Hardware and Software Requirements**

| Component | Specification / Technology | Purpose |
|---|---|---|
| Processor | Dual-core 2.0 GHz or higher | Runs the Node.js server, Vite dev server and browser |
| Memory (RAM) | 4 GB minimum, 8 GB recommended | Development tools and testing in the browser |
| Storage | About 1 GB free space | Source code and node_modules dependencies |

| Component | Specification / Technology | Purpose |
|---|---|---|
| Operating system | Windows 10/11, Linux or macOS (developed and tested on Windows 10 Pro) | Development and runtime platform |
| Runtime | Node.js 18 or later with npm (tested with Node.js 20.19.4) | Executes the backend and build tools |
| Database | MongoDB Atlas cluster or local MongoDB 6.x or later | Persistent storage of all application data |
| Browser | Current Google Chrome, Microsoft Edge or Mozilla Firefox | Access to the web application |
| Development tools | Visual Studio Code, Git and GitHub | Coding, version control and collaboration |
| Frontend libraries | React 18, React Router 6, Axios 1.6, Recharts 2.10, react-hot-toast 2.4, react-icons 5, Tailwind CSS 3.4, Vite 4.5 | User interface, routing, API calls, charts and styling |
| Backend libraries | Express 5, Mongoose 8, jsonwebtoken 9, bcryptjs 3, express-validator 7, Multer 2, csv-parser 3, helmet 8, cors, morgan, dotenv, nodemon | REST API, security, validation, uploads and logging |
| Network | Internet access when using MongoDB Atlas | Connectivity to the cloud database |

## 3.4 System Architecture

Testora follows a three-tier architecture, shown in Fig. 3.1. The **presentation tier** is a React single-page application served by Vite; it contains the public pages, the three role portals, an authentication context, protected routes and an Axios client that attaches the JWT to every request and redirects to the login page when the server answers 401. The **application tier** is an Express REST API: each request passes through security middleware, authentication and role authorization, validation chains and then a controller, which uses Mongoose models and, for analytics, a small service module. The **data tier** is a MongoDB database holding nine collections.

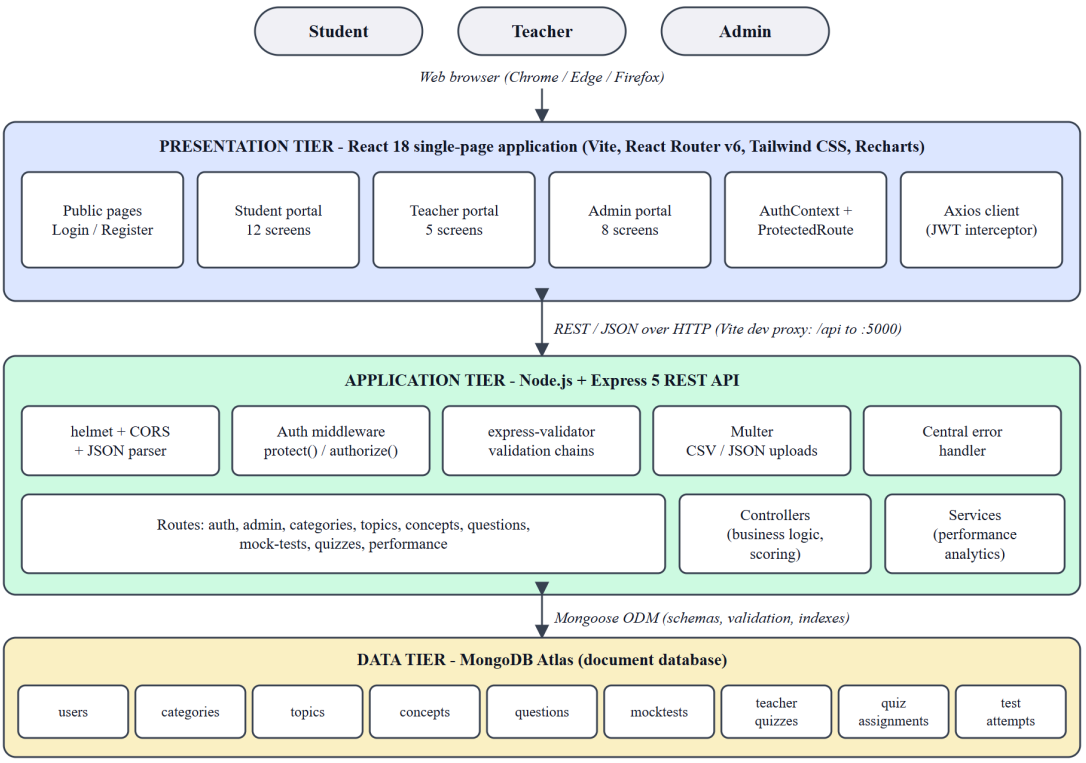



**Fig. 3.1 System Architecture of Testora**

During development the Vite server proxies every request beginning with /api to the Express server on port 5000, so the browser always talks to a single origin. In production the built static files can be served from any web server while the API runs on a Node.js host.

## 3.5 Data Flow / UML / Use Case / ER Diagram

### 3.5.1 Use Case Diagrams

Fig. 3.2, Fig. 3.3 and Fig. 3.4 show the use cases of the three actors. Every actor must first log in; the student additionally registers a new account, while teachers are created only by an administrator.

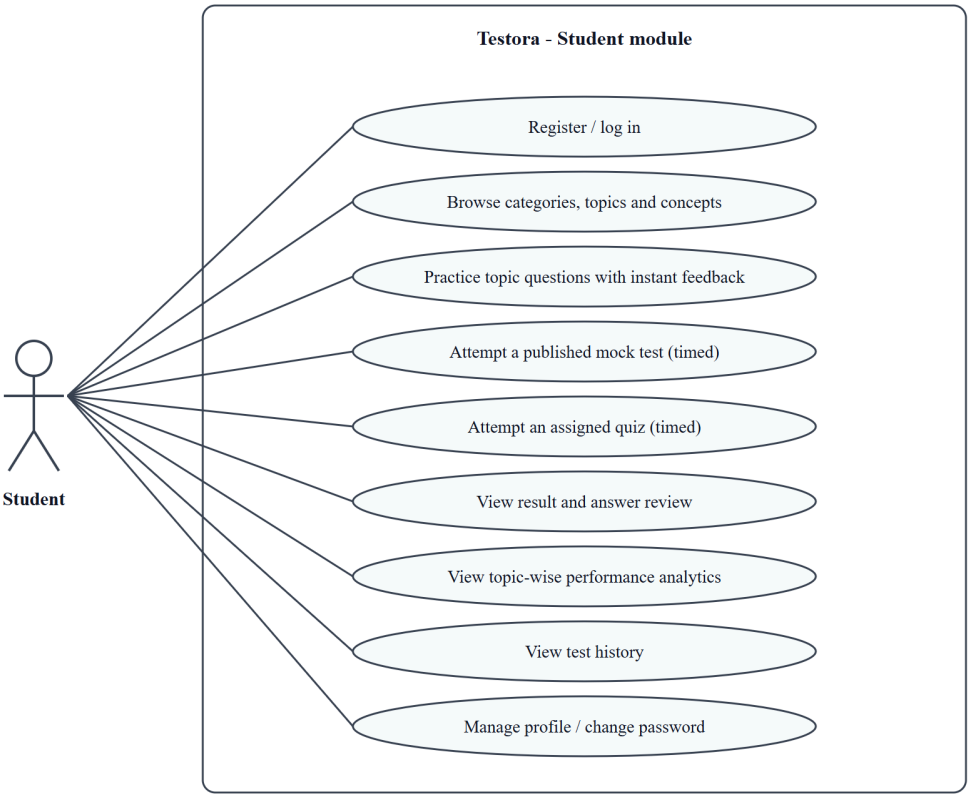



**Fig. 3.2 Use Case Diagram – Student Module**

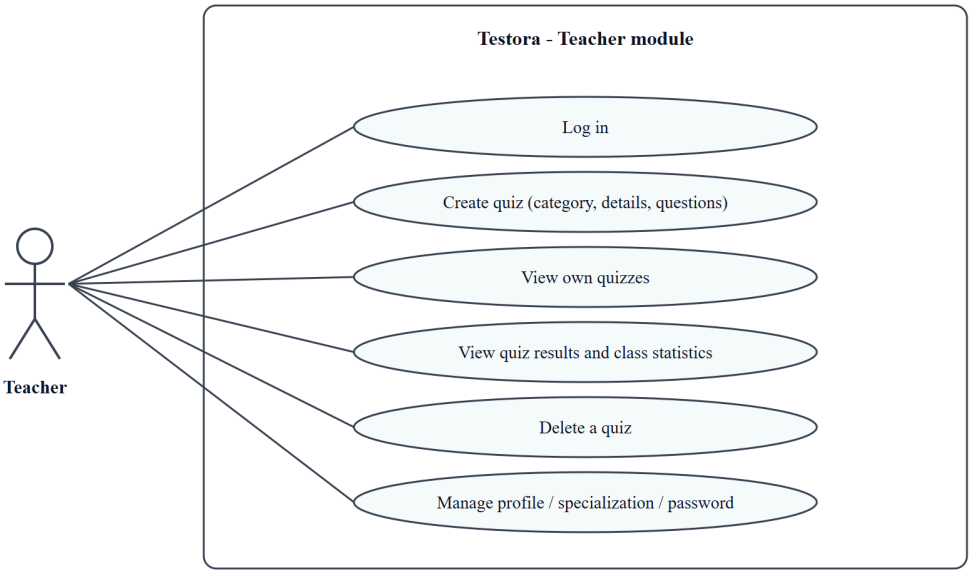



**Fig. 3.3 Use Case Diagram – Teacher Module**

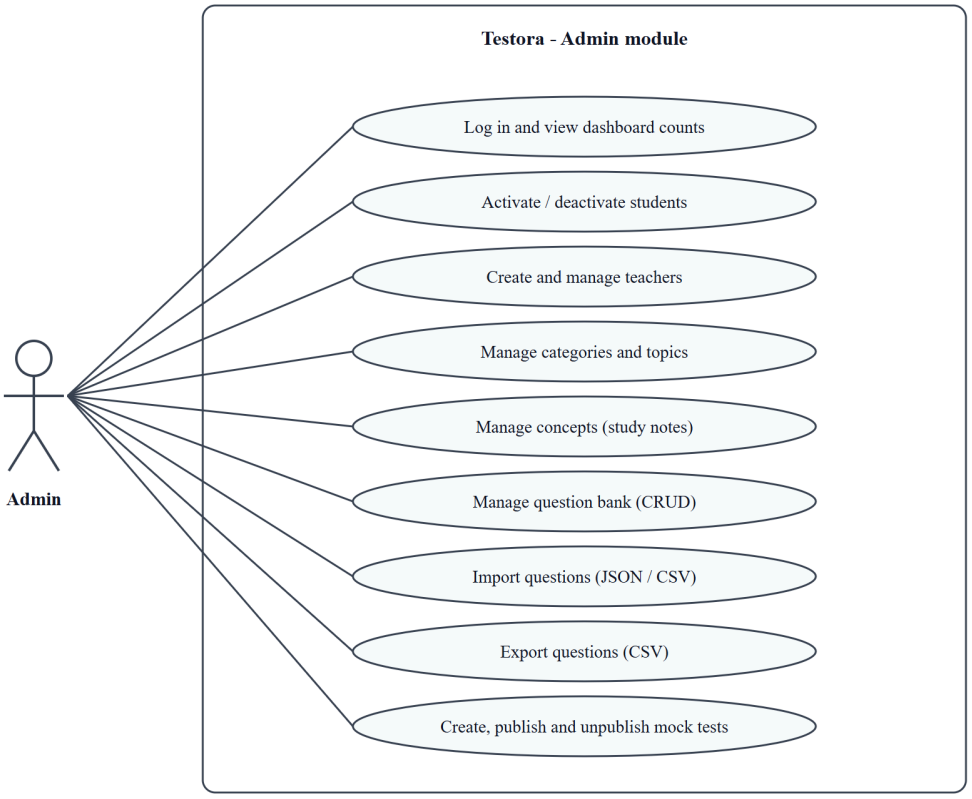



**Fig. 3.4 Use Case Diagram – Admin Module**

### 3.5.2 Data Flow Diagrams

The context diagram in Fig. 3.5 shows Testora as a single process exchanging data with the three external entities. Fig. 3.6 decomposes it into six processes and four data stores: authentication and user management (1.0), content management (2.0), mock test management (3.0), quiz management (4.0), test attempt and evaluation (5.0) and results and analytics (6.0).

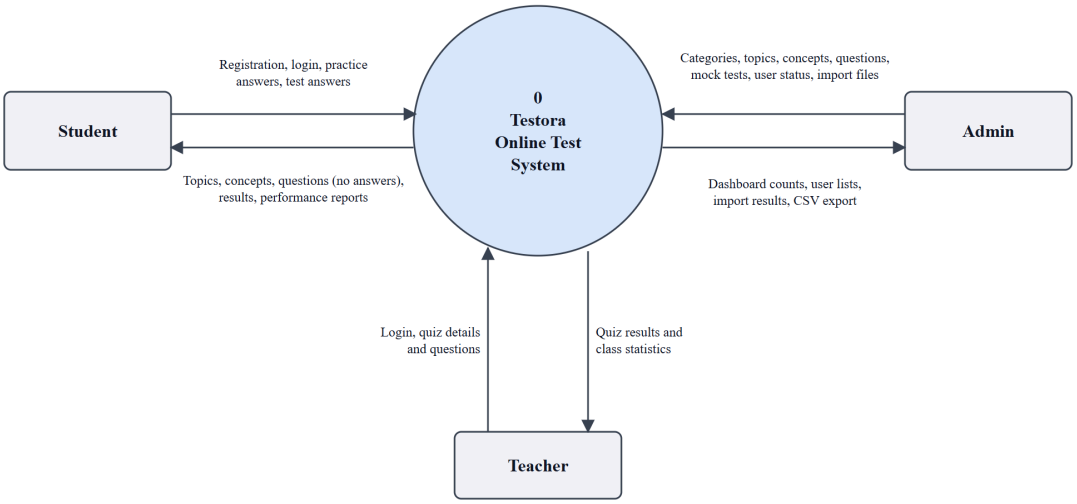
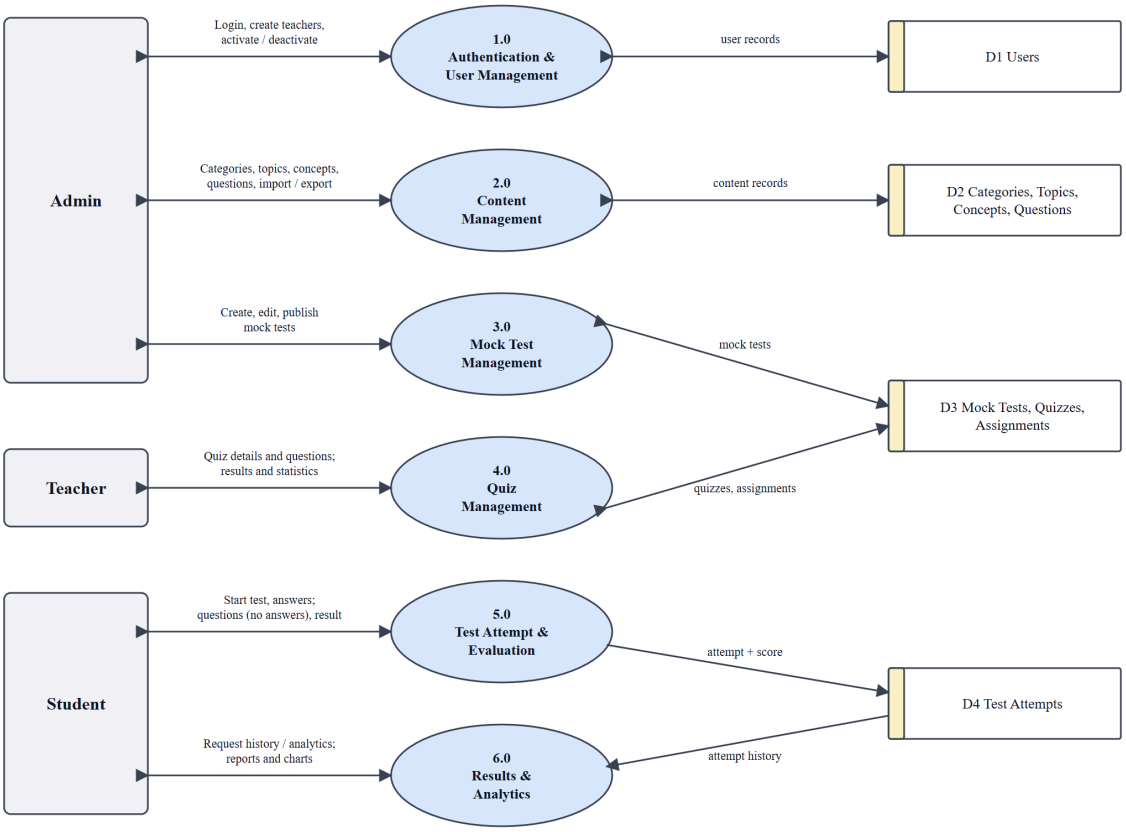



**Fig. 3.5 Data Flow Diagram – Level 0 (Context Diagram)**




*Processes 2.0, 4.0 and 5.0 also read D2 / D3 where required (arrows omitted for clarity).*

**Fig. 3.6 Data Flow Diagram – Level 1**

### 3.5.3 Entity-Relationship Diagram

Fig. 3.7 shows the nine collections and their relationships. A category has many topics; a topic has many concept notes and many questions; mock tests and teacher quizzes each reference many questions; a teacher quiz has many assignments, one per student; and a user has many test attempts.

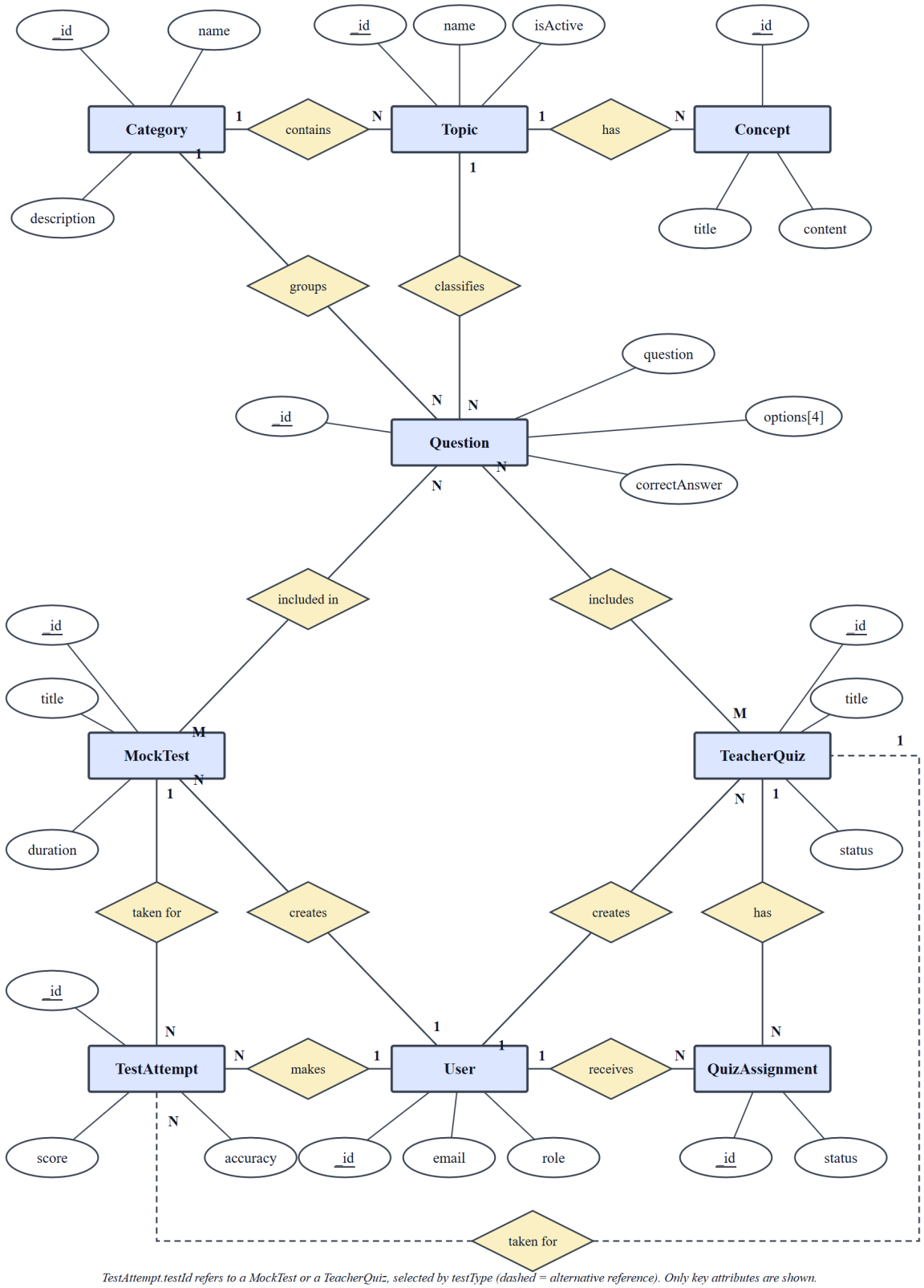



**Fig. 3.7 Entity-Relationship Diagram of Testora**

## 3.6 Database Design

Testora uses MongoDB, so each entity is a collection of documents. The primary key of every collection is the automatically generated `_id` (ObjectId) and foreign keys are stored as ObjectId references that Mongoose can populate. Every collection also has `createdAt`

and `updatedAt` timestamps. Unique indexes are defined on `User.email`, `Category.name` and on the pair (`quizId`, `studentId`) of `QuizAssignment`, which guarantees one assignment per quiz per student. Table 3.4 to Table 3.11 describe the collections.

**Table 3.4 User Collection**

| Field | Type | Constraints / Description |
|---|---|---|
| firstName, lastName | String | Required, trimmed |
| email | String | Required, unique, lower-cased |
| mobile | String | Optional 10-digit mobile number |
| passwordHash | String | Required; bcrypt hash, removed from every JSON response |
| role | String | student \| teacher \| admin (default student) |
| college, course, year | String | Student details |
| specialization | String | Teacher's area of specialization |
| profilePhoto | String | Default empty |
| isActive | Boolean | Default true; false blocks all access |

**Table 3.5 Category and Topic Collections**

| Field | Type | Constraints / Description |
|---|---|---|
| Category.name | String | Required, unique |
| Category.description, isActive | String, Boolean | isActive default true (soft delete) |
| Topic.categoryId | ObjectId | Required; reference to Category |
| Topic.name, description | String | name required |
| Topic.isActive | Boolean | Default true (soft delete) |

**Table 3.6 Concept Collection**

| Field | Type | Constraints / Description |
|---|---|---|
| topicId | ObjectId | Required; reference to Topic |
| title | String | Required |
| content | String | Explanation text |
| formulas, examples, importantPoints | [String] | Lists shown on the topic detail page |
| isActive | Boolean | Default true |

**Table 3.7 Question Collection**

| Field | Type | Constraints / Description |
|---|---|---|
| categoryId | ObjectId | Required; reference to Category |
| topicId | ObjectId | Required; reference to Topic |
| question | String | Required, trimmed |
| options | [{label, text}] | Four options labelled A–D (enforced by validation) |
| correctAnswer | String | Required; one of A, B, C, D |
| explanation | String | Optional; default empty |

| Field | Type | Constraints / Description |
|---|---|---|
| isActive | Boolean | Default true; false hides the question |
| createdBy | ObjectId | Reference to User (admin or teacher) |

**Table 3.8 MockTest Collection**

| Field | Type | Constraints / Description |
|---|---|---|
| title, description | String | title required |
| questionIds | [ObjectId] | References to Question |
| durationMinutes | Number | Required, default 30 |
| marksPerQuestion | Number | Required, default 1 |
| negativeMarksPerQuestion | Number | Required, default 0 (0 = no negative marking) |
| isPublished | Boolean | Default false; students see only published tests |
| createdBy | ObjectId | Reference to User (admin) |

**Table 3.9 TeacherQuiz Collection**

| Field | Type | Constraints / Description |
|---|---|---|
| title, description | String | title required |
| categoryId | ObjectId | Required; reference to Category |
| questionIds | [ObjectId] | References to Question |
| durationMinutes | Number | Required, default 20 |
| marksPerQuestion, negativeMarksPerQuestion | Number | Defaults 1 and 0 |
| startDate, endDate | Date | Required; the quiz can be started only inside this window |
| createdBy | ObjectId | Required; reference to User (teacher) |
| status | String | draft \| published \| expired (default draft) |

**Table 3.10 QuizAssignment Collection**

| Field | Type | Constraints / Description |
|---|---|---|
| quizId | ObjectId | Required; reference to TeacherQuiz |
| studentId | ObjectId | Required; reference to User (student) |
| assignedBy | ObjectId | Required; reference to User (teacher) |
| status | String | assigned \| started \| completed \| expired |
| assignedAt, completedAt | Date | assignedAt defaults to the current time |
| (quizId, studentId) | Unique index | One assignment per quiz per student |

**Table 3.11 TestAttempt Collection**

| Field | Type | Constraints / Description |
|---|---|---|
| userId | ObjectId | Required; reference to User (student) |
| testType, testId | String, ObjectId | mockTest \| teacherQuiz; id of the MockTest or TeacherQuiz |
| testTitle | String | Copied at start so history survives edits |
| answers | [Object] | questionId, topicId, categoryId, selectedAnswer (null = skipped), correctAnswer, isCorrect |
| score, maxMarks | Number | Marks obtained and maximum possible marks |

| Field | Type | Constraints / Description |
|---|---|---|
| totalQuestions | Number | Required |
| correctAnswers, wrongAnswers, skippedAnswers | Number | Counts |
| accuracy | Number | Percentage = correct ÷ (correct + wrong) |
| startedAt, submittedAt, timeTakenSeconds | Date, Number | Timing information |
| topicPerformance | [Object] | Per-topic attempted, correct and accuracy |
| status | String | inProgress \| completed \| expired |

## 3.7 Module Design

The application is divided into ten modules. Each module has a clear responsibility, is reachable only by the roles shown, and communicates with other modules through the REST API and the shared database. Table 3.12 summarises them and Table 4.4 in Chapter 4 lists their inputs and outputs.

**Table 3.12 Module Overview**

| Module | Responsibility | Roles | Screens / API prefix |
|---|---|---|---|
| Authentication & Authorization | Register, login, token verification, role checks, deactivation check | All roles | /api/auth; Login, Register, ProtectedRoute |
| User Management | List students and teachers, create teachers, activate or deactivate accounts | Admin | /api/admin; Students, Teachers |
| Content Management | Categories, topics, concept notes and the question bank | Admin | /api/categories, /topics, /concepts, /questions |
| Import / Export | JSON import, CSV import, CSV export of questions | Admin | /api/questions/import-json, /import, /export |
| Mock Test Management | Create, edit, publish and delete timed mock tests with marking scheme | Admin | /api/mock-tests; Mock Tests screen |
| Quiz Management | Three-step quiz creation, automatic assignment, quiz list, results | Teacher | /api/quizzes; Create Quiz, Quiz Results |
| Test Attempt & Evaluation | Start attempt, serve questions without answers, timer, submit, evaluate and score | Student | /api/mock-tests/:id/start\|submit, /api/quizzes/:id/start\|submit |
| Practice & Learning | Topic browsing, concept notes, untimed practice with instant feedback | Student | /api/questions/practice; Topics, Practice |
| Performance Analytics | Overview, topic-wise accuracy, weak-topic detection, history, result review | Student | /api/performance; Performance, History, Result |
| Common UI Services | Layout, loading and empty states, confirmation dialog, toasts, API client | All roles | DashboardLayout, ConfirmDialog, api.js |

**Authentication and authorization** verifies the token, loads the user and rejects the request if the account is missing or deactivated, then compares the user's role with the roles allowed for the route. **User management** and **content management** give the administrator control over accounts and subjects. **Import/export** moves questions in and out of the system in bulk. **Mock test** and **quiz management** define the tests; **test attempt and evaluation** runs them and produces the score; **practice and learning** and **performance**

**analytics** give students feedback; and the **common UI services** keep the interface consistent.

# CHAPTER 4
# IMPLEMENTATION

## 4.1 Development Environment

The system was developed in a local environment with the frontend and the backend running as two separate Node.js processes and a MongoDB Atlas database. Table 4.1 summarises the environment.

**Table 4.1 Development Environment**

| Item | Details |
|---|---|
| Operating system | Windows 10 Pro |
| Runtime | Node.js 20.19.4 with npm |
| Code editor / VCS | Visual Studio Code; Git with a GitHub repository |
| Database | MongoDB Atlas cloud cluster (Mongoose ODM 8) |
| Frontend toolchain | React 18, React Router 6, Tailwind CSS 3.4, Vite 4.5 (development server on port 5173) |
| Backend | Express 5 started with nodemon on port 5000; configuration read from a .env file |
| Test tooling | Python 3 (requests) for API tests; Playwright with Google Chrome for the browser walkthrough; vite build for build verification |

## 4.2 Module-wise Implementation

### 4.2.1 Authentication and Session Handling

Registration is open only to students: the controller ignores any role sent in the request and always stores `role: 'student'`. Teachers are created only by an administrator. At login the server finds the user by e-mail, rejects deactivated accounts, compares the password with the stored bcrypt hash and returns a JWT whose payload contains only the user id; the token lifetime is read from `JWT_EXPIRE` (7 days by default). The React `AuthContext` keeps the token and user in `localStorage`, re-validates the session with `GET /api/auth/me` on start-up, and `ProtectedRoute` redirects visitors without a session to the login page and users with the wrong role to their own dashboard. The Axios client adds the `Authorization` header to every request and, on any 401 response, clears the stored session and redirects to the login page.

On the server, the `protect` middleware verifies the token and then reloads the user from the database, so a deactivated account is rejected on its very next request even if its token has not expired. The `authorize` middleware then compares the role with the roles allowed for the route.

*Excerpt: server/middleware/auth.js*

```
const protect = async (req, res, next) => {
  let token;

  if (req.headers.authorization && req.headers.authorization.startsWith('Bearer')) {
    token = req.headers.authorization.split(' ')[1];
  }

  if (!token) {
    return res.status(401).json({ success: false, message: 'Not authorized, no token'
        });
  }

  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    const user = await User.findById(decoded.id);

    if (!user) {
      return res.status(401).json({ success: false, message: 'User not found' });
    }

    if (!user.isActive) {
      return res.status(401).json({ success: false, message: 'Account is deactivated'
          });
    }
```

### 4.2.2 User Management

The administrator's student list is paginated and searchable, and the toggle endpoint flips the `isActive` flag. Because `protect` checks this flag on each request, deactivation takes effect immediately (verified by test case TC11). Teachers are created with name, e-mail, mobile, password and specialization and are validated by `express-validator`. The dashboard obtains its counts in one `Promise.all` of `countDocuments` calls, so the numbers always reflect the database.

### 4.2.3 Content Management

Categories, topics, concepts and questions use the same pattern of route, validator, controller and model. Deleting a category, topic or question sets `isActive` to false instead of removing the document, so past quizzes and attempts that refer to it remain valid. The question list accepts `categoryId`, `topicId`, a case-insensitive text search and paging parameters. The raw question list, which contains correct answers, is reachable only by administrators and teachers; students receive questions only through the practice, mock test and quiz endpoints, which remove `correctAnswer` and `explanation`.

### 4.2.4 Import and Export

The JSON importer first validates every item (question text, four options, a correct answer of A–D, an existing category and an existing topic that belongs to that category) and checks for duplicates inside the file and against the database. If any item fails, the whole request is rejected with a list of item numbers and reasons and nothing is saved; only a fully valid batch is inserted with a single `insertMany`. The uploaded file is limited to 2 MB and to the

`.json` type by Multer, and an import may contain at most 500 questions. The CSV importer reports row-level errors but imports the valid rows.

CSV export builds a header row and one row per question and quotes every value that contains a comma, a double quote or a line break, doubling embedded quotes, as required by the CSV format specification [18]. The export accepts the same `categoryId`, `topicId` and `search` filters as the question list, so either all or filtered questions can be exported.

### 4.2.5 Mock Test Management

The administrator selects questions from the bank (optionally filtered by category), sets the duration, the marks per question and an optional negative mark and saves the test. A test is invisible to students until it is published; publishing is a partial update that changes only `isPublished`, which is why the update route validates only the fields that were sent.

### 4.2.6 Teacher Quiz Creation

The Create Quiz page is a wizard with three steps. Step 1 loads the categories created by the administrator, so any new category appears immediately. Step 2 collects the quiz details and validates the dates and numeric limits. When the teacher continues, step 3 generates as many question forms as the chosen question count (1 to 50). On submission the client sends one request containing the quiz details and all questions. The server validates the payload, confirms that the category exists and that every selected topic belongs to it, inserts the questions with `insertMany`, creates the quiz with the new question ids, and the client then calls the assignment endpoint with `assignAll: true`.

### 4.2.7 Test Attempt and Evaluation

Starting a test checks the rules for that test type: a mock test must be published and not already completed; a quiz must be assigned to the student, published and inside its start–end window. A new attempt with status `inProgress` is stored and the questions are returned without `correctAnswer` and `explanation`. On submission the server loads the questions with their correct answers, compares each submitted answer, counts correct, wrong and skipped answers, calculates the marks, builds the topic-wise performance and saves the completed attempt. The correct answers are copied into the attempt, so the review stays accurate even if a question is edited later.

*Excerpt: server/controllers/mockTestController.js (submitMockTest)*

```
    test.questionIds.forEach((q) => {
      questionMap[q._id.toString()] = q;
    });

    const processedAnswers = [];
    let correct = 0;
    let wrong = 0;
    let skipped = 0;
    // ...
```

```
    const marksPerQuestion = test.marksPerQuestion || 1;
    const negativeMarksPerQuestion = test.negativeMarksPerQuestion || 0;
    const obtainedMarks = Math.round((correct * marksPerQuestion - wrong *
        negativeMarksPerQuestion) * 100) / 100;
    const maxMarks = Math.round(test.questionIds.length * marksPerQuestion * 100) / 100;
```

### 4.2.8 Practice and Learning

The topic detail page reads the topic, its concept cards and the question count from a single endpoint. Practice mode loads the topic's questions without answers and checks each answer through `POST /api/questions/practice/submit`, which returns whether the answer is correct, the correct option and the explanation. Practice answers are not stored as attempts.

### 4.2.9 Performance Analytics

Every completed attempt stores a per-topic summary. The analytics endpoints add these summaries over all of a student's attempts, compute the accuracy for each topic and classify it with the function below. Topics with fewer than five attempted questions are shown as "Not Enough Data".

### 4.2.10 Common Interface Components

Shared components keep the three portals consistent: a role-aware `DashboardLayout` with a collapsible sidebar, `LoadingSpinner` and `EmptyState` components, toast notifications from react-hot-toast and a promise-based `ConfirmDialog` hook that replaces the browser's `confirm()` for destructive actions such as deleting a quiz or deactivating a user.

## 4.3 Algorithms / Business Logic

### 4.3.1 Test Evaluation and Scoring

Fig. 4.1 shows the life cycle of an attempt. The scoring rule is applied on the server for both mock tests and quizzes:

*Pseudocode: scoring*

```
for each question q in the test:
    selected = answer submitted for q, or null
    if selected is null:            skipped = skipped + 1
    else if selected == q.correct:  correct = correct + 1
    else:                           wrong   = wrong + 1

score     = correct x marksPerQuestion  -  wrong x negativeMarksPerQuestion
maxMarks  = numberOfQuestions x marksPerQuestion
accuracy  = round(100 x correct / (correct + wrong))    // 0 if nothing attempted
```

Skipped questions never reduce the score. For example, with 2 marks per question and a negative mark of 0.5, one correct and one wrong answer in a two-question test give 2 − 0.5 = 1.5 marks out of 4 (test cases TC23 and TC27).

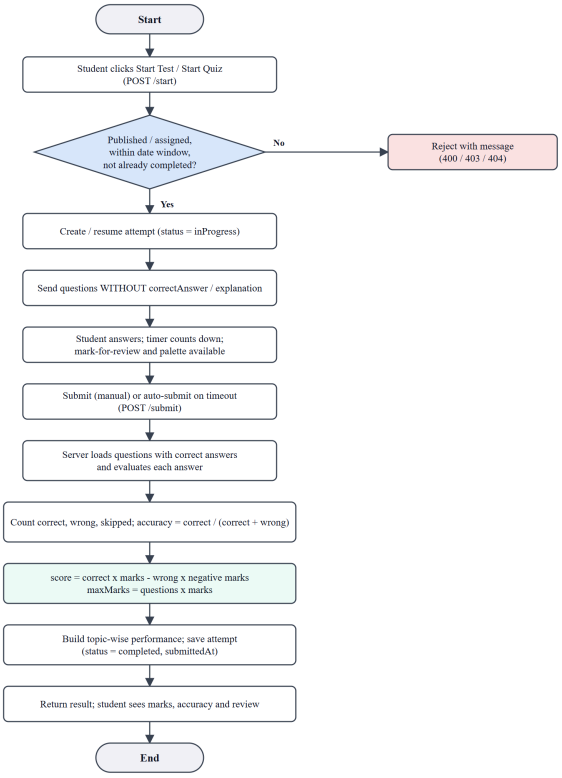



**Fig. 4.1 Test Attempt and Evaluation Flowchart**

### 4.3.2 Topic Classification

After at least five attempted questions in a topic, the aggregate accuracy decides its status, as shown in Table 4.2.

**Table 4.2 Topic Classification Rules**

| Condition (aggregate over all attempts) | Status shown to the student |
|---|---|
| Fewer than 5 attempted questions | Not Enough Data |
| Accuracy below 60 % | Weak |
| Accuracy from 60 % to 80 % | Needs Practice |
| Accuracy above 80 % | Strong |

### 4.3.3 All-or-Nothing JSON Import

*Pseudocode: JSON import*

```
errors = []; seen = {}
for each item i in questions:
    check question text, options A-D, correctAnswer in {A,B,C,D}
    check category exists and topic exists and topic belongs to category
    key = category + topic + lower-case(question)
    if key already in seen: error "duplicate in file"
    otherwise remember key
if errors is not empty:  reject whole request (400) with errors; save nothing
if any key exists in database: reject whole request (400); save nothing
insertMany(all items)    // only reached when the whole batch is valid
```

### 4.3.4 Quiz Creation and Assignment

*Pseudocode: quiz creation*

```
validate request fields (express-validator)
verify category exists; verify each topic exists and belongs to the category
questions  = insertMany(question forms)
quiz       = create TeacherQuiz(categoryId, questionIds = questions._id, marks, dates)
assign     : for every active student  upsert QuizAssignment(quiz, student)
             and set quiz.status = published
```

## 4.4 User Interface / Screens

The main screens of each portal are shown below. All screenshots were captured from the running application with the demonstration data set.

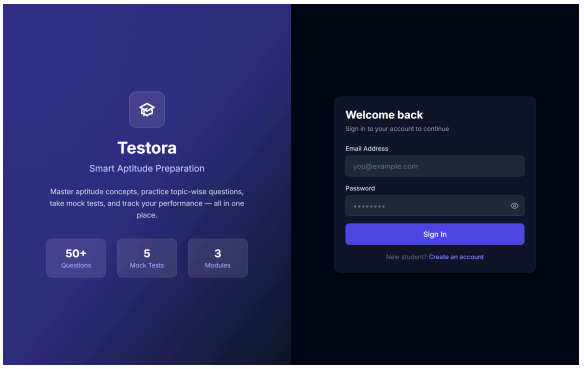



**Fig. 4.2 Login Screen**

The login screen validates that both fields are filled and shows the server's message (for example "Invalid credentials" or "Account is deactivated. Contact admin.") as a notification. After a successful login the user is redirected to the dashboard of their own role.

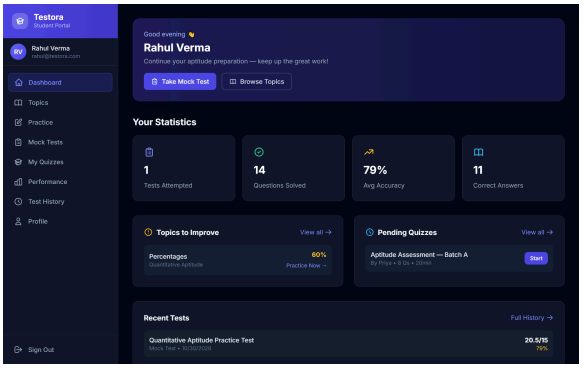



**Fig. 4.3 Student Dashboard**

The student dashboard summarises tests attempted, questions solved, average accuracy and correct answers, lists topics to improve with a direct "Practice Now" link, shows pending quizzes and the most recent results.

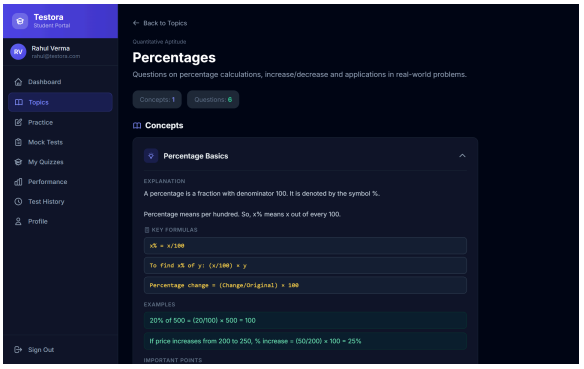



**Fig. 4.4 Topic Detail with Concept Cards**

A topic page shows its description, the number of concepts and questions and expandable concept cards containing the explanation, formulas, worked examples and important points.

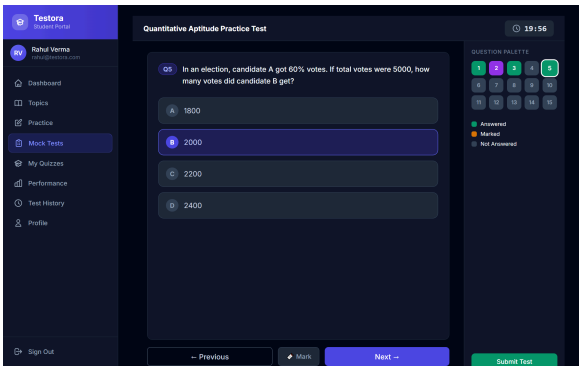



**Fig. 4.5 Timed Exam Screen with Question Palette**

The exam screen shows a countdown timer, one question at a time, navigation buttons, a mark-for-review button and a palette in which answered, marked and unanswered questions have different colours.

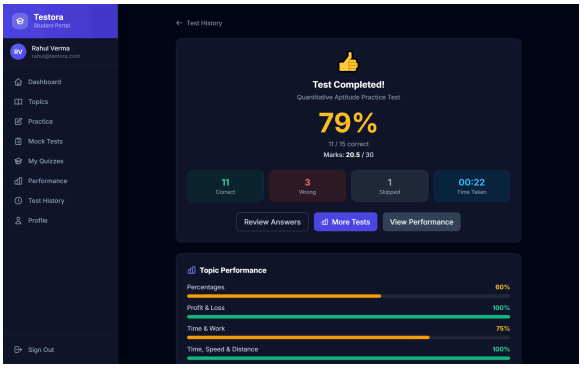



**Fig. 4.6 Result Screen**

After submission the result screen shows accuracy, the number of correct, wrong and skipped answers, the time taken, the marks obtained out of the maximum (here 20.5 out of 30 with negative marking) and the topic-wise performance.

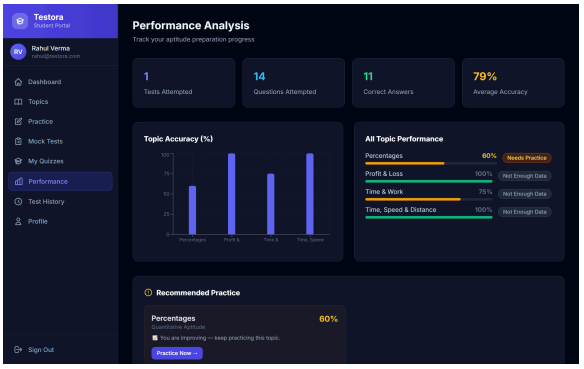



**Fig. 4.7 Performance Analysis**

The performance screen aggregates all completed attempts into overall statistics, a topic-accuracy chart, a classification of each topic and a list of recommended practice topics.

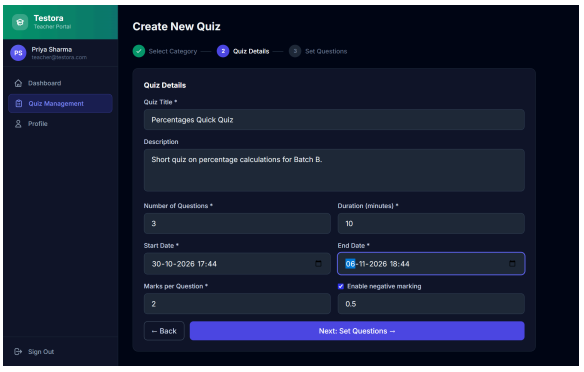



**Fig. 4.8 Create Quiz – Step 2: Quiz Details**

In step 2 the teacher enters the title, description, number of questions, duration, marks per question, optional negative marking and the start and end date-time of the quiz.

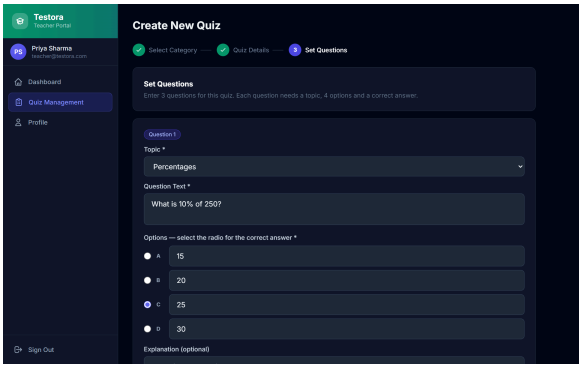



**Fig. 4.9 Create Quiz – Step 3: Set Questions**

Step 3 generates one form for each requested question. Each form requires a topic, the question text, four options, the correct option and optionally an explanation; submitting creates the questions, creates the quiz and assigns it to all active students.

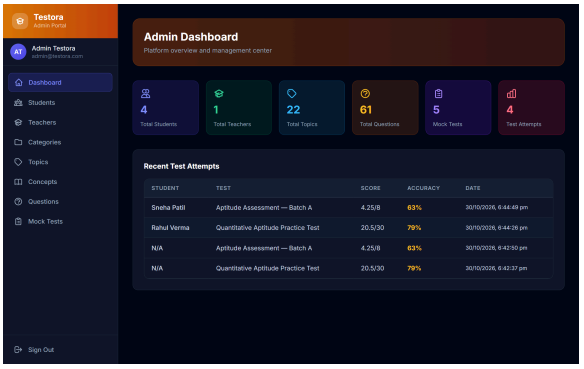



**Fig. 4.10 Admin Dashboard**

The admin dashboard shows live database counts in the order Total Students, Total Teachers, Total Topics and Total Questions, followed by mock test and attempt counts and the most recent attempts.

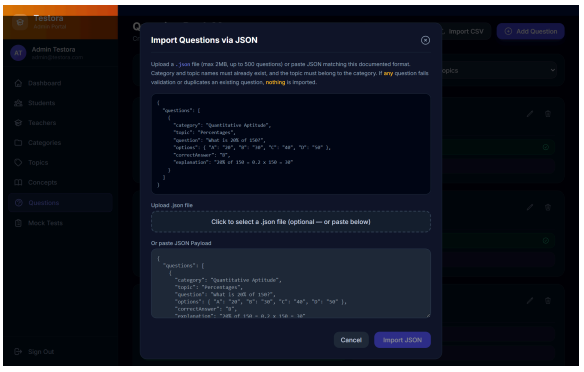



**Fig. 4.11 Admin – Import Questions via JSON**

The import dialog documents the JSON format, accepts a .json file of up to 2 MB or pasted JSON and explains that nothing is imported if any question fails validation or is a duplicate.

## 4.5 Integration and Deployment

The client and server are integrated through the REST API. In development, the Vite server forwards requests whose path starts with `/api` to the Express server, so the React code simply calls relative URLs such as `/api/auth/login`. The Express server connects to MongoDB through Mongoose when it starts and reads its settings from environment variables, listed in Table 4.3. The values shown are the placeholders from the `.env.example` file; real secrets must never be committed.

**Table 4.3 Environment Variables of the Server**

| Variable | Purpose | Example / placeholder |
|---|---|---|
| PORT | Port of the Express server | 5000 |
| MONGODB_URI | MongoDB connection string (local or Atlas) | mongodb://localhost:27017/testora |
| JWT_SECRET | Secret used to sign and verify tokens (must be long and private) | your_super_secret_jwt_key_change_in_production |
| JWT_EXPIRE | Token lifetime | 7d |

| Variable | Purpose | Example / placeholder |
|---|---|---|
| CLIENT_URL | Browser origin allowed by CORS | http://localhost:5173 |
| NODE_ENV | Environment name | development |

The steps to run the system are: (1) install dependencies in the `server` and `client` folders with `npm install`; (2) copy `.env.example` to `.env` and set the values; (3) optionally load the demonstration data with `npm run seed`, which first clears the target database; (4) start the API with `npm run dev` in `server`; (5) start the client with `npm run dev` in `client` and open `http://localhost:5173`. A production build of the client is produced with `npm run build` and is a set of static files that can be hosted on any web server, while the API can be run with `npm start` on any Node.js host; `CLIENT_URL` must then be set to the public address of the client. The server creates the `uploads` folder used for temporary import files when it starts. No public deployment was carried out for this mini project; the complete installation guide is given in Appendix A.

## 4.6 Security and Validation

Fig. 4.12 shows the path of a request through the security controls of the API.

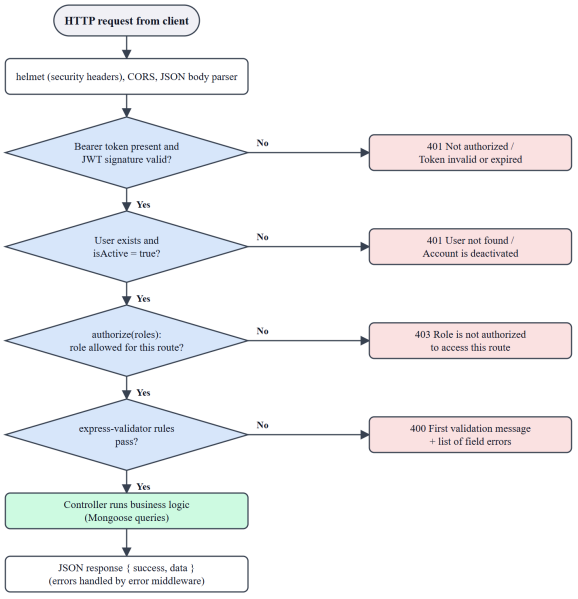



**Fig. 4.12 Request Authentication and Authorization Pipeline**

- **Authentication.** Passwords are hashed with bcrypt (10 salt rounds) before storage and the hash is removed from every JSON response. Tokens are signed with `JWT_SECRET`.
- **Authorization.** Every route except registration, login and the health check calls `protect`, and routes that belong to one role also call `authorize`. A student cannot read the question bank, create a quiz or open the admin dashboard (test cases TC06 to TC09).

- **Role protection.** The registration endpoint always stores the role `student`, so a user cannot make themselves an administrator by adding a role to the request (TC04).
- **Answer secrecy.** Question payloads sent before submission never include `correctAnswer` or `explanation` (TC22, TC31); the review is available only from the stored attempt of the same student (TC29, TC30).
- **Input validation.** `express-validator` chains validate registration, login, teachers, categories, topics, questions, quizzes (including 1–50 questions, valid dates and numeric marks) and mock tests; failures return HTTP 400 with the first message and a list of field errors.
- **File upload control.** Multer limits CSV files to 5 MB and JSON files to 2 MB and 500 questions, accepts only the `.json` type for JSON, and the temporary file is deleted after it has been read.
- **Transport and headers.** `helmet` sets secure HTTP headers, CORS allows only the configured client origin and the JSON body size is limited.
- **Exception handling.** Controllers catch errors and a central error handler converts Mongoose cast, duplicate-key and validation errors and upload errors into clear JSON messages with suitable status codes.
- **Data protection.** Content is soft-deleted (`isActive`) so history is preserved, and deactivated accounts are blocked immediately.

**Table 4.4 Module Inputs and Outputs**

| Module | Purpose | Input | Output |
|---|---|---|---|
| Authentication | Create and verify user sessions | E-mail, password, registration details, token | Signed JWT, user profile, 401/403 errors |
| User Management | Control accounts and teacher records | Student or teacher id, teacher details | Updated user record, status message |
| Content Management | Maintain categories, topics, concepts, questions | Names, descriptions, question text, options, correct answer | Stored records, filtered question lists |
| Import / Export | Bulk load and export questions | JSON body or .json file, CSV file, filters | Import summary or error list, CSV download |
| Mock Test Management | Define and publish timed tests | Title, question ids, duration, marks, negative marks | Mock test record, published state |
| Quiz Management | Create and assign teacher quizzes | Category, details, question forms, dates | Quiz, new questions, assignments, results table |
| Test Attempt & Evaluation | Run and score attempts | Test id, answers, time taken | Attempt with score, maxMarks, counts, topic performance |
| Practice & Learning | Learn and practise by topic | Topic id, selected answer | Concept cards, correctness, explanation |
| Performance Analytics | Summarise results | Student’s completed attempts | Overview, topic accuracy, weak topics, history |
| Common UI Services | Shared interface behaviour | User actions | Layout, dialogs, toasts, spinners |

# CHAPTER 5
# TESTING & RESULTS

## 5.1 Testing Strategy

Testing combined four activities, following the usual progression from component checks to system-level validation [16]:

- **API functional testing.** A script of 41 test cases called the REST API with valid and invalid inputs for every role and recorded the actual response of each call. The script was run against a separate demonstration database seeded with the sample data, so the working data was never modified.
- **Browser walkthrough.** A scripted Google Chrome session logged in as a student, a teacher and an administrator and used the real screens: a student took a mock test and another student took a quiz through the "Start" buttons, a teacher created a quiz through the three-step wizard and an administrator opened every management screen. The screenshots in Chapter 4 come from this session.
- **Security testing.** Role restrictions, answer secrecy, role injection at registration and immediate blocking of deactivated users were tested explicitly.
- **Build and responsiveness checks.** The client was compiled with `vite build` and the main screen was captured at a phone-sized viewport (390 × 844).

No automated unit-test framework or load-testing tool was used; this is listed under the limitations in Section 5.5.

## 5.2 Test Cases

Table 5.1 lists the executed test cases with the actual result recorded by the script.

**Table 5.1 Test Cases and Results**

| Test Case ID | Module | Input / Condition | Expected Result | Actual Result | Status |
|---|---|---|---|---|---|
| TC01 | Authentication | Valid admin e-mail and password | 200, JWT token and role = admin | 200, token issued, role = admin | Pass |
| TC02 | Authentication | Valid e-mail, wrong password | 401 Invalid credentials | 401, "Invalid credentials" | Pass |
| TC03 | Authentication | Empty login payload | 400 with validation message | 400, "Valid email is required" | Pass |
| TC04 | Authentication | Register with extra field role = "admin" | 201 and role forced to student | 201, role = student | Pass |
| TC05 | Authentication | Protected route without token | 401 Not authorized | 401, "Not authorized, no token" | Pass |
| TC06 | Authorization | Student calls GET /admin/dashboard | 403 Forbidden | 403, "Role 'student' is not authorized to access this route" | Pass |
| TC07 | Authorization | Student calls GET /questions (answer bank) | 403 Forbidden (answers not exposed) | 403 | Pass |
| TC08 | Authorization | Student calls POST /quizzes | 403 Forbidden | 403 | Pass |

| Test Case ID | Module | Input / Condition | Expected Result | Actual Result | Status |
|---|---|---|---|---|---|
| TC09 | Authorization | Teacher calls GET /admin/teachers | 403 Forbidden | 403 | Pass |
| TC10 | Admin dashboard | Compare dashboard counts with list endpoints | Counts equal database records | students 5/5, topics 22/22, questions 61/61 | Pass |
| TC11 | Student management | Deactivate student; reuse existing token and try login | 401 for old token and for new login | token: 401 "Account is deactivated"; login: 401 "Account is deactivated. Contact admin." | Pass |
| TC12 | Teacher management | Admin creates teacher with specialization | 201 and specialization saved | 201, specialization = Logical Reasoning | Pass |
| TC13 | Teacher management | Create teacher with invalid data | 400 validation error | 400, "First name is required" | Pass |
| TC14 | Categories | Create category with empty name | 400 Category name is required | 400, "Category name is required" | Pass |
| TC15 | Categories | Create category (admin) | 201 and category saved | 201, name = Data Interpretation | Pass |
| TC16 | Quiz creation | End date earlier than start date | 400 End date must be after start date | 400, "End date must be after start date" | Pass |
| TC17 | Quiz creation | Topic does not belong to selected category | 400 topic/category mismatch | 400, "Topic 'Percentages' does not belong to the selected category" | Pass |
| TC18 | Quiz creation | Quiz with zero questions | 400 validation error | 400, "Between 1 and 50 questions are required" | Pass |
| TC19 | Quiz creation | Valid quiz, 2 inline questions, 2 marks, -0.5 | 201; 2 new questions saved; marks stored | 201; questions 61->63; marks 2/-0.5 | Pass |
| TC20 | Quiz assignment | Assign with assignAll = true | Assigned to every active student | 200, 4 assignments / 4 active students | Pass |
| TC21 | Quiz attempt | Start a quiz before its start date | 400 Quiz has not started yet | 400, "Quiz has not started yet" | Pass |
| TC22 | Quiz attempt | Start assigned quiz; inspect question payload | Questions returned without correctAnswer / explanation | 200, 2 questions, answer fields present = False | Pass |
| TC23 | Scoring (quiz) | 1 correct + 1 wrong, 2 marks each, -0.5 negative | score = 1.5, maxMarks = 4 | score = 1.5, maxMarks = 4, accuracy = 50% | Pass |
| TC24 | Quiz results | Teacher opens results of the quiz | Completed student with marks obtained / max marks | Arjun: 1.5/4 | Pass |
| TC25 | Mock tests | Partial update: isPublished = true | 200 and only that field changes | 200, isPublished = True, title kept = TC Mock | Pass |
| TC26 | Mock tests | Update with invalid isPublished | 400 validation error | 400, "isPublished must be true or false" | Pass |
| TC27 | Scoring (mock test) | 1 correct + 1 wrong, 2 marks each, -0.5 negative | score = 1.5, maxMarks = 4 | score = 1.5, maxMarks = 4 | Pass |
| TC28 | Mock tests | Start the same mock test a second time | 400 already attempted | 400, "You have already attempted this test" | Pass |
| TC29 | Results | Student opens own result after submission | Review shows correct answers only after submission | 200, correctAnswer in review = B | Pass |
| TC30 | Results | Another student requests this attempt | 404 (attempts are private) | 404 | Pass |
| TC31 | Practice | Load practice questions for a topic | No answer fields before answering | 200, 12 questions, answer fields present = False | Pass |
| TC32 | Practice | Submit one practice answer | Feedback with isCorrect, correctAnswer, explanation | 200, keys = ['correctAnswer', 'explanation', 'isCorrect'] | Pass |
| TC33 | JSON import | Valid JSON body (1 question) | 201, 1 question imported | 201, "Imported 1 questions successfully." | Pass |
| TC34 | JSON import | Re-import the same question | 400, rejected as duplicate | 400, "Import rejected: 1 question(s) already exist in the question bank. No questions were imported." | Pass |

| Test Case ID | Module | Input / Condition | Expected Result | Actual Result | Status |
|---|---|---|---|---|---|
| TC35 | JSON import | Batch of 2: one valid, one missing option D | 400, nothing imported (no partial import) | 400, question count 65 -> 65; error: Missing option D | Pass |
| TC36 | JSON import | Unknown category name | 400, category not found | 400, "Category 'Nope' not found" | Pass |
| TC37 | JSON import | Upload a .txt file | 400 Only .json files are allowed | 400, "Only .json files are allowed" | Pass |
| TC38 | JSON import | Upload a 3797 KB .json file (limit 2 MB) | 400 File too large | 400, "File too large" | Pass |
| TC39 | JSON import | 501 questions in one import (limit 500) | 400 import limit exceeded | 400, "Import limit exceeded: a single import supports at most 500 ..." | Pass |
| TC40 | CSV export | Export question containing comma, quotes and newline | CSV with header; special characters quoted/escaped (RFC 4180) | 200, text/csv; charset=utf-8, escaped correctly = True | Pass |
| TC41 | CSV export | Export filtered by category | Only that category's rows exported | 16 rows exported / 16 questions in category | Pass |

## 5.3 Results / Output

All 41 test cases passed (41 of 41). Table 5.2 shows the results by module.

**Table 5.2 Test Results by Module**

| Module | Test Cases | Passed |
|---|---|---|
| Authentication | 5 | 5 |
| Authorization | 4 | 4 |
| Admin dashboard | 1 | 1 |
| Student management | 1 | 1 |
| Teacher management | 2 | 2 |
| Categories | 2 | 2 |
| Quiz creation | 4 | 4 |
| Quiz assignment | 1 | 1 |
| Quiz attempt | 2 | 2 |
| Scoring (quiz) | 1 | 1 |
| Quiz results | 1 | 1 |
| Mock tests | 3 | 3 |
| Scoring (mock test) | 1 | 1 |
| Results | 2 | 2 |
| Practice | 2 | 2 |
| JSON import | 7 | 7 |
| CSV export | 2 | 2 |
| **Total** | **41** | **41** |

In the browser walkthrough every screen of the three portals loaded with data, the student completed a mock test (11 correct, 3 wrong, 1 skipped in a 15-question test with 2 marks per question and a negative mark of 0.5, giving 20.5 out of 30 marks) and a quiz, the teacher’s quiz results showed the completed student’s marks (for example 4.25 out of 8 for

5 correct and 3 wrong answers at 1 mark and a negative mark of 0.25), and the production build of the client completed successfully. Table 5.3 compares the outcome with the objectives set in Chapter 1.

**Table 5.3 Objectives and Outcome**

| No. | Objective (summary) | Outcome |
|---|---|---|
| 1 | Role-based portals with JWT and server-side role checks | Achieved – TC01 to TC09 passed; three portals implemented |
| 2 | Category → Topic → Question hierarchy with admin-managed content | Achieved – demonstration data: 3 categories, 22 topics, 58 questions, 5 mock tests |
| 3 | Server-side evaluation, marks, negative marking, answer secrecy | Achieved – TC22, TC23, TC27, TC29 to TC32 passed |
| 4 | Three-step quiz creation with dynamic forms and auto-assignment | Achieved – TC16 to TC20 passed; screens in Fig. 4.8 and Fig. 4.9 |
| 5 | Topic-wise analytics with Weak / Needs Practice / Strong | Achieved – performance screen verified in the browser walkthrough |
| 6 | Validated JSON import, CSV export and at least 40 test cases | Achieved – 41 test cases executed, TC33 to TC41 cover import and export |

## 5.4 Error Handling and Validation Results

Table 5.4 shows how the system responds to invalid input and exceptional situations; the test case numbers refer to Table 5.1.

**Table 5.4 Handling of Invalid Inputs and Exceptions**

| Situation | Observed system behaviour |
|---|---|
| Wrong password at login | 401 with the message “Invalid credentials”; the user stays on the login page (TC02) |
| Empty or malformed input | 400 with the first validation message and the list of field errors (TC03, TC13, TC14, TC18) |
| Quiz end date before start date | 400 “End date must be after start date”; nothing is saved (TC16) |
| Topic from a different category | 400 “Topic … does not belong to the selected category” (TC17) |
| Deactivated account | Existing token and new login are both rejected with “Account is deactivated” (TC11) |
| Access to another role’s route | 403 “Role … is not authorized to access this route” (TC06 to TC09) |
| Starting a quiz too early / test twice | 400 “Quiz has not started yet” and “You have already attempted this test” (TC21, TC28) |
| Invalid or duplicate JSON import | 400 with an item-wise error list or a duplicate message; question count unchanged (TC34 to TC36) |
| Wrong file type, too large file, too many items | 400 “Only .json files are allowed”, “File too large”, “Import limit exceeded” (TC37 to TC39) |

## 5.5 Limitations

- **Resuming a mock test is not supported.** If a student reloads the page while a mock test is in progress, the start endpoint returns the stored attempt record instead of the question set, so the exam screen cannot be restored and the timer is not recovered.

Quiz attempts do not have this problem. This is the most important defect found and is the first item of the future work.

- **No automated test suite in the repository.** The test script used for this report ran outside the project and is not part of it; there are no unit or integration tests and no continuous integration.
- **Password rule is enforced only in the browser.** The registration form requires at least one upper-case letter, but the server enforces only a minimum length of six characters.
- **Session handling is simple.** The token is kept in `localStorage`, there is no refresh token, rate limiting, account lockout, e-mail verification or password reset; the 7-day token lifetime is mitigated only by the per-request active-user check.
- **Limited question model.** Only single-answer multiple-choice questions with four options are supported, question order is not randomised and each mock test or quiz can be attempted once.
- **CSV and JSON imports differ.** The CSV importer imports valid rows and reports invalid ones, whereas the JSON importer is all-or-nothing.
- **Scope of testing.** The interface was tested in Chrome only, the load and concurrency behaviour was not measured, and the system was not deployed to a public server.
- **Teachers and students cannot be deleted**, only deactivated, by design.

# CHAPTER 6
# CONCLUSION & FUTURE SCOPE

## 6.1 Conclusion

Students preparing for aptitude examinations lacked one integrated platform that the college could control, and teachers lacked a simple way to set and monitor timed tests. This project delivered *Testora*, a role-based web application in which administrators manage users, subjects, questions and mock tests, teachers create and assign quizzes through a guided workflow, and students learn concepts, practise, take timed tests and receive marks-based results with topic-wise feedback.

The objectives set in Chapter 1 were met. The three portals are secured with JWT authentication and server-side role checks, content is organised as Category → Topic → Question, tests are evaluated on the server with configurable marks and negative marking without exposing correct answers, quiz creation generates question forms dynamically and assigns the quiz automatically to all active students, performance is analysed topic by topic, and questions can be imported from validated JSON and exported to CSV. All 41 documented test cases passed and the complete user flows were demonstrated in a browser.

The work also showed the importance of validating on the server, of deciding explicitly which data a student may see, and of testing role boundaries as carefully as normal functions. The known limitations, in particular the missing resume support for in-progress mock tests, are listed in Section 5.5.

## 6.2 Future Scope

- Restore an in-progress mock test after a page reload, including the remaining time, and expire abandoned attempts automatically.
- Add PDF export or printing of question sets and result sheets.
- Randomise question and option order, add question difficulty levels and more question types such as multiple-answer and numeric answers.
- Add proctoring aids such as full-screen enforcement and tab-switch detection, and allow re-attempts under teacher control.
- Add an automated regression suite (unit, API and end-to-end tests) and a continuous-integration pipeline.
- Strengthen account security with refresh tokens, rate limiting, e-mail verification, password reset and server-side enforcement of the password strength rule.

- Add notifications (e-mail or in-app) when a quiz is assigned, leaderboards and downloadable analytics for teachers.
- Reduce the size of the client bundle with route-based code splitting and deploy the application to a public host with HTTPS.

# REFERENCES


[1] Meta Platforms, Inc., “React documentation,” React. Accessed: [DD-MMM-2026]. [Online]. Available: https://react.dev/

[2] OpenJS Foundation, “Express – Node.js web application framework,” Express.js. Accessed: [DD-MMM-2026]. [Online]. Available: https://expressjs.com/

[3] MongoDB, Inc., “MongoDB manual,” MongoDB Documentation. Accessed: [DD-MMM-2026]. [Online]. Available: https://www.mongodb.com/docs/manual/

[4] Mongoose contributors, “Mongoose documentation,” Mongoose ODM. Accessed: [DD-MMM-2026]. [Online]. Available: https://mongoosejs.com/docs/

[5] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” Internet Engineering Task Force, RFC 7519, May 2015. [Online]. Available: https://www.rfc-editor.org/rfc/rfc7519

[6] N. Provos and D. Mazières, “A future-adaptable password scheme,” in Proc. USENIX Annual Technical Conf. (FREENIX Track), Monterey, CA, USA, 1999, pp. 81–91.

[7] Tailwind Labs, “Tailwind CSS documentation,” Tailwind CSS. Accessed: [DD-MMM-2026]. [Online]. Available: https://tailwindcss.com/docs

[8] Vite contributors, “Vite – Getting started,” Vite. Accessed: [DD-MMM-2026]. [Online]. Available: https://vite.dev/guide/

[9] Recharts contributors, “Recharts – a redefined chart library built with React and D3,” Recharts. Accessed: [DD-MMM-2026]. [Online]. Available: https://recharts.org/

[10] express-validator contributors, “express-validator documentation,” express-validator. Accessed: [DD-MMM-2026]. [Online]. Available: https://express-validator.github.io/docs/

[11] Express.js contributors, “Multer – Node.js middleware for handling multipart/form-data,” GitHub. Accessed: [DD-MMM-2026]. [Online]. Available: https://github.com/expressjs/multer

[12] Moodle Pty Ltd, “Quiz activity,” MoodleDocs. Accessed: 30-Oct-2026. [Online]. Available: https://docs.moodle.org/en/Quiz_activity

[13] Google LLC, “Create & grade quizzes with Google Forms,” Google Docs Editors Help. Accessed: 30-Oct-2026. [Online]. Available: https://support.google.com/docs/answer/7032287

[14] IndiaBIX Technologies, “Aptitude questions and answers,” IndiaBIX. Accessed: 30-Oct-2026. [Online]. Available: https://www.indiabix.com/aptitude/questions-and-answers/

[15] Testbook Edu Solutions Pvt. Ltd., “Testbook.com – India’s No.1 Govt Exam Preparation Site | Online Course | Mock Test,” Testbook. Accessed: 30-Oct-2026. [Online]. Available: https://testbook.com/

[16] I. Sommerville, Software Engineering, 10th ed. Harlow, U.K.: Pearson Education, 2016.

[17] OWASP Foundation, “OWASP Top 10:2021,” OWASP. Accessed: [DD-MMM-2026]. [Online]. Available: https://owasp.org/Top10/

[18] Y. Shafranovich, “Common format and MIME type for comma-separated values (CSV) files,” Internet Engineering Task Force, RFC 4180, Oct. 2005. [Online]. Available: https://www.rfc-editor.org/rfc/rfc4180

# APPENDICES

## Appendix A – User Manual / Installation Guide

**Prerequisites.** Node.js 18 or later with npm, Git, and either a MongoDB Atlas account or a local MongoDB server.

1. Get the source code: `git clone https://github.com/oskhamkar/Testora` and open the folder.
2. Backend: `cd server`, run `npm install`, copy `.env.example` to `.env` and set `MONGODB_URI`, `JWT_SECRET` (a long random value), `JWT_EXPIRE`, `PORT` and `CLIENT_URL` (see Table 4.3).
3. Optional demonstration data: run `npm run seed` in `server`. This clears the target database first and then creates the demonstration users, 3 categories, 22 topics, 5 concepts, 58 questions, 5 mock tests and one teacher quiz.
4. Start the API: `npm run dev` in `server`. The message "Testora server running on port 5000" and "MongoDB Connected" confirm that it is ready; `GET /api/health` returns a JSON status.
5. Frontend: `cd client`, run `npm install` and `npm run dev`, then open `http://localhost:5173` in a browser.

**Demonstration accounts** (created by the seed script; change the passwords before any real use):

**Table A.1 Demonstration Accounts**

| Role | E-mail | Password |
|---|---|---|
| Admin | admin@testora.com | Admin@123 |
| Teacher | teacher@testora.com | Teacher@123 |
| Student | rahul@testora.com | Student@123 |
| Student | sneha@testora.com | Student@123 |

**Using the system.** *Student:* register or log in, open Topics to read concepts, use Practice for untimed questions, take a published mock test from Mock Tests or an assigned quiz from My Quizzes, then open the result for the answer review and use Performance to see weak topics. *Teacher:* open Quiz Management, choose Create Quiz, select a category, enter the details, fill in the generated question forms and submit; open Results on a quiz to see the class. *Administrator:* use the sidebar to manage students, teachers, categories, topics, concepts, questions (including Import JSON, Import CSV and Export CSV) and mock tests; use the eye button on a mock test to publish or unpublish it.

## Appendix B – Test Evidence

Table B.1 shows the test cases grouped by module; the full list with inputs and actual results is in Table 5.1.

**Table B.1 Test Evidence by Module**

| Module | Test case IDs |
|---|---|
| Authentication | TC01, TC02, TC03, TC04, TC05 |
| Authorization | TC06, TC07, TC08, TC09 |
| Admin dashboard | TC10 |
| Student management | TC11 |
| Teacher management | TC12, TC13 |
| Categories | TC14, TC15 |
| Quiz creation | TC16, TC17, TC18, TC19 |
| Quiz assignment | TC20 |
| Quiz attempt | TC21, TC22 |
| Scoring (quiz) | TC23 |
| Quiz results | TC24 |
| Mock tests | TC25, TC26, TC28 |
| Scoring (mock test) | TC27 |
| Results | TC29, TC30 |
| Practice | TC31, TC32 |
| JSON import | TC33, TC34, TC35, TC36, TC37, TC38, TC39 |
| CSV export | TC40, TC41 |

## Appendix C – Supporting Documents

No supporting documents such as an internship or company confirmation, survey or questionnaire, publication or certificate, or third-party API permission were required for this mini project, so this appendix is not applicable. The information about existing systems in Chapter 2 is taken from the public web pages listed in the References.